第一章　ウォーキングアップ

2021年11月9日　　20:28

1_1.　nCrの計算（計算途中でのオーバーフローを防ぐため）

練習問題１　Hornerの方法

1_3.例題３　個別の得点が全体で何位かを求める。一つずつ調べる。

1_3_1 改良版　juni[0]~juni[101]の行列を用いる

1_3_2 負数の場合

1_6　ユークリッドの互除法　m-nの代わりに剰余（ｍ％ｎ）を用いる

1_7_1 エラトステネスのふるい　素朴な方法、計算回数n√n/2

1_7_2 小さな素数から順にその倍数を「ふるい」から一括で落とす。

第二章　数値計算

2021年11月9日　　22:14

2_2 _1 数値積分 台形則

Sn＝$\frac{h}{2}\{f(x_k)+f(x_{k+1})\}$

2_2_2 シンプソン則　シンプソン則では，微小区間上端の曲線を 2次曲線で近似する ・ 積分結果はかなり簡単な式になる

(x0,f(x0)),(x1,f(x1)),(x2,f(x2))

3点を通る二次曲線ｇ（ｘ）を作る.

[x0~x2]g(x) dx= h*(f(x0) +4f(x1) + f(x2))/3

奇数項の合計f0

偶数項の合計fe

[a ~ b]f(x) dx = {f(a) + f(b) +4(f0 + f(b-h)) + 2*fe}*h/3

中点則

2_3 テイラー展開

2_4 非線形方程式の解法

2分法

ニュートン法

計算量を評価するのは、「オーダ」という

誤差には以下のものが存在する：

- 丸め誤差（桁数制限のために四捨五入する）
- 打ち切り誤差（有限回数で計算を打ち切る）
- 桁落ち（近い値同士での引き算で生じる）

第三章　ソートとサーチ

2021年12月7日　　16:37

ソート(sort:整列)

サーチ(search:探索)

	ソート法	特徴	計算量
基本形	**バブル・ソート**	隣接2データの交換を繰り返す．交換回数が多い．安定．	$O(n^2)$
	直接選択法	数列から最大 (最小) を探す．比較は多，交換は少．安定．	$O(n^2)$
	基本挿入法	ソート済み列への挿入を繰り返す．安定．	$O(n^2)$
改良形	**クイック・ソート**	データ列を大小に2分割する操作を再帰的に繰り返す．不安定．	$O(n\log n)$
	ヒープ・ソート	数列をヒープ構造というデータ構造に組み替える．不安定．	$O(n\log n)$
	シェル・ソート	データをとびとびの部分列にして基本挿入法を適用．不安定．	$O(n^{1.25})$

直接選択法

3_1_1.c

```
 for(i=0; i<N-1; i++){
        (j=i+1; j<N; j++){
                If(a[j]<min)
        }
}
```

バブル・ソート

3_1_2.c

```
for(i=0; i<N-1; i++){
        (j=N-1; j>i; j--){
                If(a[j]<a[j-1])
        }
}
```

シェーカー・ソート

3_1_3.c

基本挿入法

シェル・ソート

逐次探索法 では，データを先頭から順番に探索する方法．運良くすぐ見つかれば調べる手間は小さいが，最悪の場合は最後まで見つからず，n 回調べることになる．データ数が n であれば，要する計算量は O(n).

番兵：一番最後のデータの後ろに更に1つ余分な要素を追加し，ここにキーと同じデータを加えておくと…i と N との大小比較をしなくても，最後の余分なデータに到達すれば探索が終了してくれるので, 余分な条件判定が不要となり，若干の高速化が望める (条件判定は多少時間がかかる仕事である)． この目的で最後に追加するデータを 番兵 (sentinel) と呼ぶ.

2分探索法 は，あらかじめソートされたデータ群から探索するとき有効．データを大小2つのグループに半分に分け，境目よりも小さいなら次は小さい方のグループを，大きいなら次は大きい方のグループを捜す．計算量は O(logn)

ハッシュ(hash)

第四章　再帰

2021年12月8日　　18:48

第五章　データ構造

2021年12月15日　　10:39

①スタック(stack:棚)

後入れ先出し

②キュー(queue:待ち行列)

先入れ先出し

③リスト(list)：

メモリ領域の取得

1.入力とは逆順なリスト

```
    head=NULL;
    while(p=talloc(),scanf("%s %s,"p->name.p->
tel)!=EOF{
   p->pointer=head;
   head=p;
 }
```

2.入力順のリスト

```
  head=talloc();
  scanf();
  old=head;
 while(p=tallloc(),scanf()!=EOF){
   old->pointer=p;
   old=p;
}
 old->pointer=NULL;
```

3.ダミー・ノード

4.リストへの挿入

```
    p=head;
    While(p!=NULL){
    If( strcmp (key, p->name)==0){
        n->pointer = p->pointer;
        p->pointer = n;
         reture;
    }
    p=p->pointer;
```

```
    }
```

5.リストへの削除

```
  p=old=head;
  while(p!=NULL){
    If( strcmp (key, p->name)==0){
      if(p==head)
            head=p->pointer;
      else
            old->pointer=p->pointer;
      return;
    }
    old=p;
    p=p->pointer;
  }
```

6.リストへの削除（ダミー・ノード）

逆順入力でダミー・ノードを作成

```
p=head;
  while(p->pointer!=NULL){
    If( strcmp (key, p->pointer->name)==0){
      p->pointer=p->pointer->pointer;
        return;
    }
    p=p->pointer;
  }

  head=NULL;
 do{
  p=talloc();
  p->pointer=head;
  head=p;
}scanf();
```

メモリ領域の開放

メモリ領域の開放

- malloc() を使って動的に確保したメモリを解放してシステム (コンピュータ) に返還する関数：free()
- 例えば，ポインタ p で参照されるノードを解放するには，free(p); とすればよい．これだけで，スライド上に大きな赤のバッテンで描いた消去処理が実現できる．
- 例えば教科書 p.244 のコードでは，以下を追加すればよい．

```
if (strcmp(key,p->name)==0)
    if (p==head)
       {head=p->pointer; free(p);}
    else
       {old->pointer=p->pointer; free(p);}
    return;
}
```

赤ペンなどで
書き加えてください

お行儀の良いプログラミングを！

今までのリストを　線形リスト（linear list）と呼ぶ

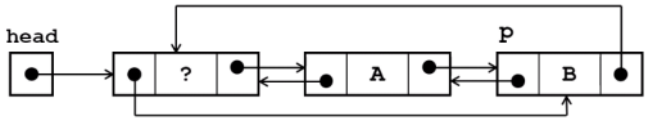
1.循環リスト(circular list)

2.双方向リスト（doubly-linked list）

3.循環・双方向リスト
　　ダミー・ノード

循環・双方向リスト

- 既存の循環・双方向リストに新たなノードを加える手順は以下のようになる (順番に注意).
  (1) p (新ノード) の right を先頭ノード (head) へ.
  (2) p の left を最後尾ノード (head->left) へ.
  (3) 最後尾ノード (head->left) の right を p へ.
  (4) 先頭ノード (head) の left を p へ.



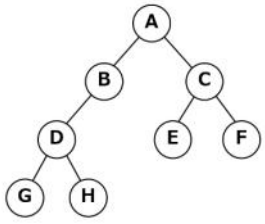
この順番を間違うと何が起こるか考えてみること

第六章　木(tree)

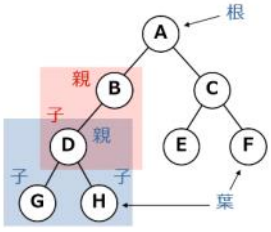
2022年1月12日　　11:08

木とは

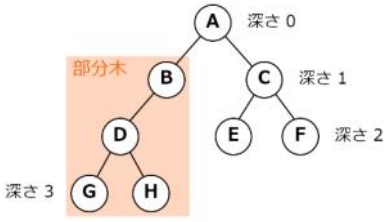
- リスト構造では枝分かれは起こらなかった．枝分かれしながらデータが伸びていくデータ構造を木 (tree) と呼ぶ．
- 木はノード (node) とそれらを結ぶ枝 (branch) から構成される．データが記録されているのはノードであり，枝はデータとデータの間の親子関係を示す．



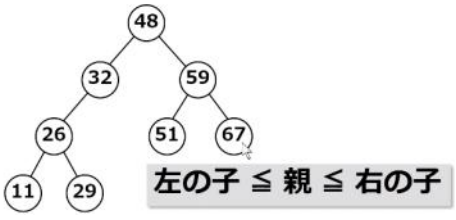
- あるノードから下に分岐した先のノードを子，分岐元のノードを親という．
- 木の一番最初のノードを特に，根 (root) と呼び，子を持たないノードを葉 (leaf) と呼ぶ．



- あるノードから下につながった木構造の全体を部分木 (subtree) という．
- 根からあるノードに到達するまでに通る枝の数をそのノードの深さ (depth) といい，根から最も深いノードまでの深さを木の高さ (height) という．



- ノードからの分岐が2以下の木を2分木 (binary tree) と呼ぶ．
- 親と子の関係が，ある規則 (大きい，小さいなど) に従って並べられている木を2分探索木 (binary search tree) という．



- 木構造はきわめて重要で，多くの実用的対象が存在する．
  - 組織データやファイルシステムなどの階層構造の表現
  - AI における論理的関係性の表現
  - 探索を容易にする情報表現 (2分探索木など)
  - ヒープ・ソート
- 講義では，2分探索木およびヒープに重点を置いて解説する．

二分探索木
O(log n) 計算量

6.1二分探索木の配列表現

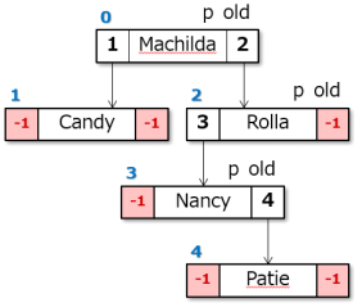
6.2二分探索木の動的表現

2分探索木へのデータの追加

```
➔ p=0;
  while (p!=nil){
➔   old=p;
    if (strcmp(key,a[p].name)<=0)
➔     p=a[p].left;
    else
➔     p=a[p].right;
  }

➔ a[sp].left=a[sp].right=nil;
➔ strcpy(a[sp].name,key);
  if (strcmp(key,a[old].name)<=0)
➔   a[old].left=sp;
  else
➔   a[old].right=sp;
➔ sp++;
```

sp: 5
key: "Patie"
p: -1



6.3(p.286)二分探索木の再帰的表現

ノード追加の再帰的方法

※ ここでは，データ Nancy を含むノードへのポインタを [Nancy] と表記する．

第１引数が NULL 以外：
・適切な子を起点とする再帰呼び出し
・第一引数をオウム返し

第１引数が NULL ：
・新規ノードを作成
・新規ノードへのポインタを返す

20

ノード追加の再帰的方法 (ソース)

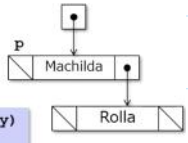
```
struct tnode *gentree(struct tnode *p, char *w)
{
    if (p == NULL){
        p = talloc();
        strcpy(p->name, w);
        p->left = p->right = NULL;
    }
    else if (strcmp(w, p->name) < 0)
        p->left = gentree(p->left, w);
    else
        p->right = gentree(p->right, w);
    return p;
}
```

6.3(p.287)二分探索木のサーチ

2分探索木のサーチ (再帰版)

```
while (printf("Search name -->"),
       scanf("%s",key)!=EOF){
  if ((p=search(root,key))!=NULL)
    printf("%s が見つかりました¥n",p->name);
  else
    printf("見つかりません¥n");
}
```

key: "Rolla"




```
struct tnode *search(struct tnode *p,char *key)
{
  if (p==NULL || strcmp(key,p->name)==0)
    return p;
  if (strcmp(key,p->name)<0)
    return search(p->left,key);
  else
    return search(p->right,key);
}
```

6.4(p.291)二分探索木のトラバーサル(深さ優先)

①行きがけ順

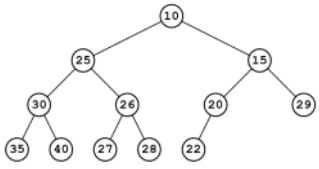
②通りがけ順　(アルファベット順)

③帰りがけ順

6.5(p.297)レベルごとのトラバーサル(試験範囲外)

6.6(p.301)ヒープ

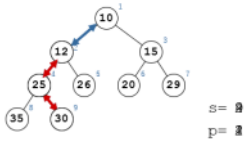
- ヒープ (heap : 山，堆積) とは，すべての親が必ず2つの子を持つ (最後の要素は左の子だけでも良い) 完全2分木で，どの親と子をとっても，親≦子になっている木を言う．



ヒープへのデータ追加(上方移動)

ヒープへのデータ追加 (上方移動)

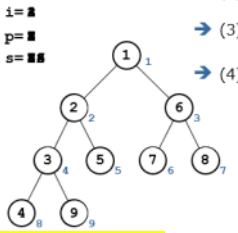
- (1) 新しいデータをヒープの最後の要素として格納する．
- (2) その要素を子とする親のデータと比較し，親の方が大きければ子と親を交換する．
- (3) 次に，その親を子とする親 (おじいちゃん) との間で比較を行い，必要があれば交換．これを，子の位置が根まで上がるか，親≦子になるまで繰り返す．



ヒープの作成(p.307)(下方移動)

ヒープの作成 (下方移動)

- 図的に理解しよう.



- (1) 子を持つ最後の親から開始してループ (最初は 9/2 = 4 から)
- (2) 親の番号を p とし，2p と 2p+1 で小さい方の子を s とする.
- (3) s が p より大きいときは (1) ループで次へ.
- (4) p と s を交換し，p=s, s=2p とする. s>n なら (1) ループで次へ. s<=n なら (2) へ.

(2) から (4) の手順を 下方移動 (shift down) と呼ぶ

ソースコード・p.307

ヒープ・ソート　p.309

ヒープ・ソート (ソースコード)

- p.309.
- 関数 `swap()` は2つの配列要素の交換.
- `main`関数内，最初の `while`ループは初期ヒープの作成. データを入れるごとに，入れたデータを最後尾に格納した上で，上方移動を使ってヒープを整える作業を繰り返している.
- 2つめの `while`ループは，データを取り出すごとのループ.
- 最初にルート (`1`番) と最後尾 (`n`番) を交換して，最後尾をヒープから除外している (`n--`).
- 次のループは，視点 `p` を根に初期化して下方移動を1回行っている部分.
- p.311 では，初期ヒープ作成を下方移動で行っている.

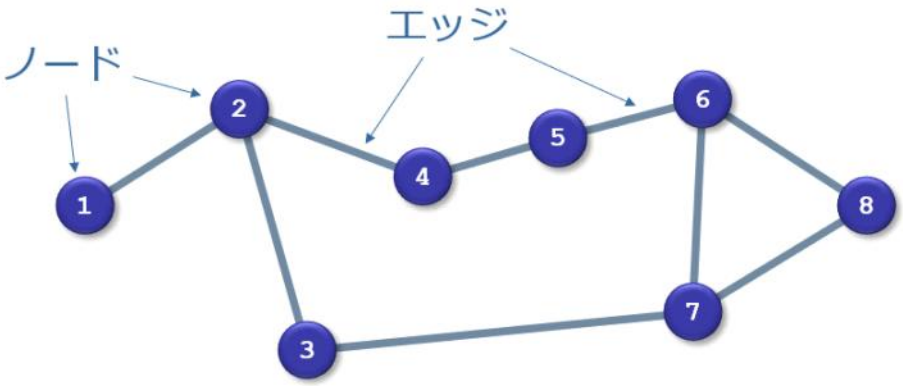
$O(n \log n)$

第七章　グラフ

2022年2月2日　　10:52

グラフとは

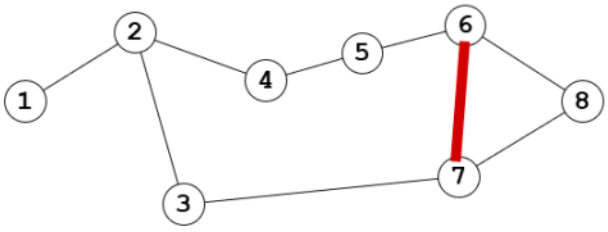
- 円で描いた部分はノード (node 節点，vertex 頂点ともいう) であり，それを結んでいるのは エッジ (edge 辺，branch 枝) である．ノード1と2はつながっているが，1と3はつながっていない．



グラフとは

- グラフをデータとして表現するときに使われるのは隣接行列 (adjacency matrix) である．ノード i と j の間に接続がある場合は 1，ない場合は 0 としておく．

教科書ではノード番号は1からとしている．

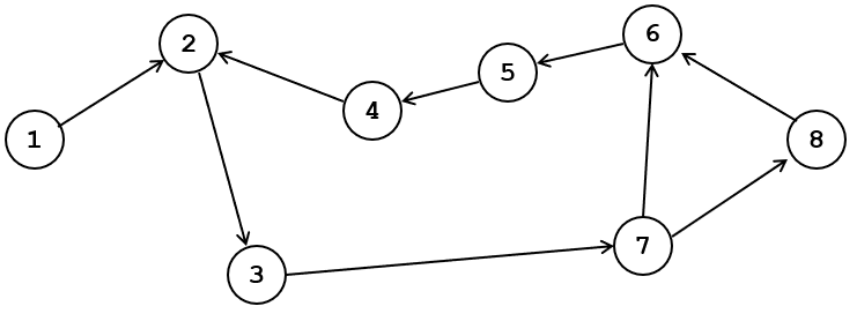


j

i	1	2	3	4	5	6	7	8
1	0	1	0	0	0	0	0	0
2	1	0	1	1	0	0	0	0
3	0	1	0	0	0	0	1	0
4	0	1	0	0	1	0	0	0
5	0	0	0	1	0	1	0	0
6	0	0	0	0	1	0	1	1
7	0	0	1	0	0	1	0	1
8	0	0	0	0	0	1	1	0

対角成分は，同じノードへの接続をあるとするかどうかに関係する (この場合はなしとしている)．

- ここまでの例では，グラフのエッジには向きはなかった．このようなものを無向グラフ (undirected graph) という．
- これに対して，「進める方向」のような向きを含んだグラフを有向グラフ (directed graph) という．この場合，隣接行列は，ノード i から j へのエッジに対して，要素 (i, j) は 1，要素 (j, i) は 0 としておく (p.333 図7.4)．



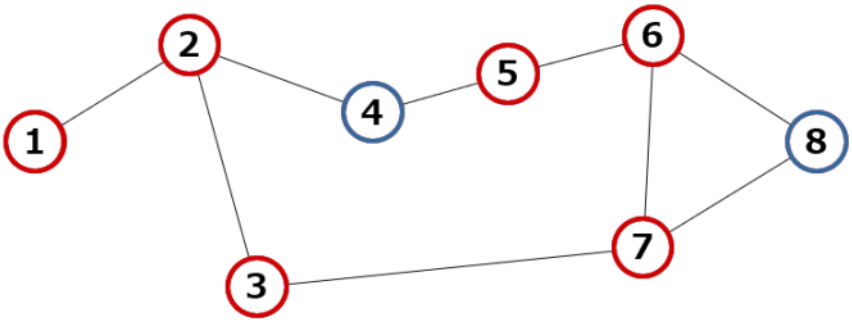
深さ優先探索

- どのように探索が進むかを図で見てみよう．

(1) 始点を出発し，ノード番号の若い順に進む位置を調べ，いけるところ (エッジで連結されていてまだ訪問していないノード) まで進む．

(2) 行き場所がなくなったら，まだいける方向の残っている分岐点まで戻り，再びいけるところまで進む．

(3) 行き場所が全てなくなったら終了 (来た道を戻る)．

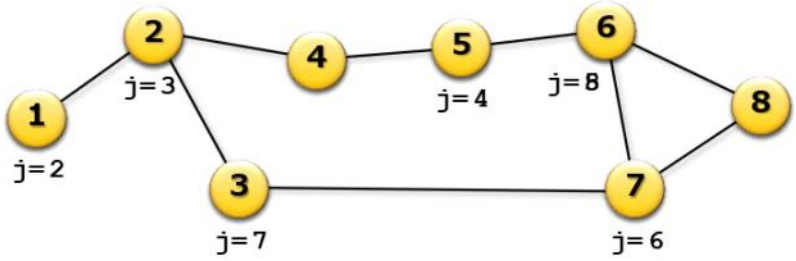


深さ優先探索の再帰的解法

- 深さ優先探索を再帰的に行う関数は右の通り.
- 未探索の行き先を順番に探して，行けるときは再帰呼び出しでそこに進む.
- visit(1)を呼ぶと…

```
void visit(int i)
{
    int j;

    v[i]=1;
    for (j=1;j<=N;j++){
        if (a[i][j]==1 && v[j]==0){
            printf("%d->%d ",i,j);
            visit(j);
        }
    }
}
```



(p.335)

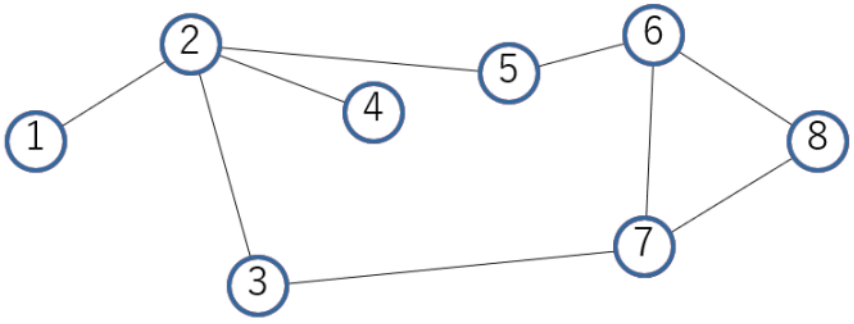
幅優先探索

(1)始点をキューに入れる.

(2)キューからノードを取り出し，訪問済みとする．そのノードに連結している未訪問ノードを全てキューに入れておく.

(3)キューが空なら終了，そうでないときは (2) へ.

ソースコード：p.340-341

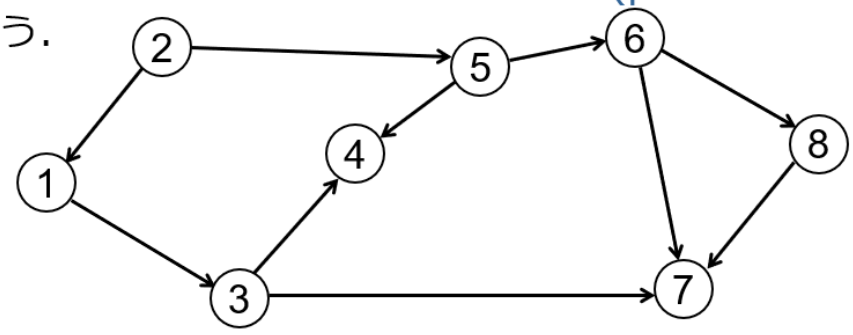


キュー： 1 2 3 4 5 7 6 8

p.340

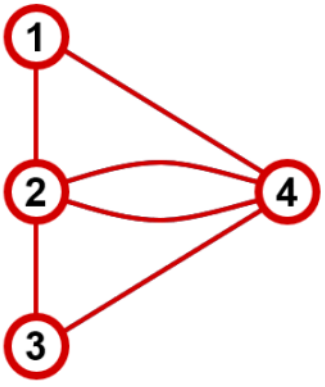
トポロジカル・ソート

- グラフを見ると，2から仕事を始めたとき，4や7ができるためには，1, 3 が出来ていなければならないのと同時に，右側では 5, 6, 8 が出来ていなければならない．教科書に誤植
- つまり，2, 1, 3, 4, 7 という系列と 2, 5, 4, 6, 8, 7 という系列がある．従って，2の次に行いうる仕事，1と5ではどちらを先に行っても良い．このように必ずしも順序を比較できない場合がある順序関係を半順序関係 (partial order) という．



Euler の一筆書き

- 辺を消しながら深さ優先探索を行い，出発点に戻った時に全ての経路が消えていれば，そのとき通った経路が求める経路である



スタック `v[n]`

1 4 2 4 3 2 1

ソースコード：p.350

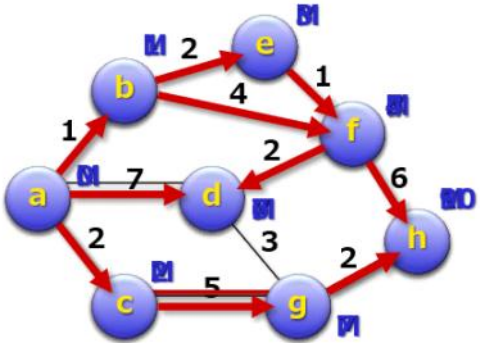
最短路問題 (Dijkstraのアルゴリズム)

- **解きたい問題：ノードaからhまでの最短距離を求める**

(1) 全てのノードの確定フラグを下ろし，距離値を無限大で初期化
(2) スタートノードの距離値を０とする．(3), (4) をＮ回繰り返す
(3) 距離値最小の未確定ノードの番号を ｐとし，確定フラグを立てる
(4) ｐにつながる全ノードについて，ｐから進んだときの距離値が現在の距離値より小さくなるとき，そのノードの距離値を更新する

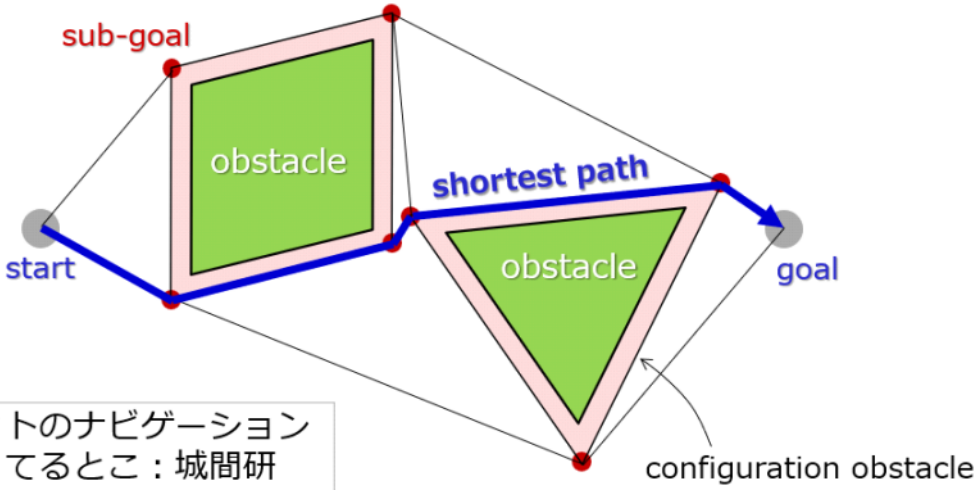
- あるノードの距離値を更新するとき，距離値だけでなく「この距離値になる場合，直前のノードはこれ」という「手前ノード情報」も記録する

→ 距離だけでなく経路も求まる



最短路問題の例

- 障害物環境下での移動ロボットのナビゲーション
- コンフィギュレーション障害物の頂点をサブゴールとし，**可視グラフ (visibility graph)** を構築，最短路を探索



移動ロボットのナビゲーションを主にやってるとこ：城間研